

Fundamentos de Git y GitHub

Módulo: Fundamentos de desarrollo Front-End

|AE7: Gestionar el código fuente de un proyecto utilizando github para mantener un repositorio de código remoto seguro y permitir trabajo concurrente.

Introducción

En este módulo trabajaremos Git, como sistema de control de versiones distribuido y GitHub como plataforma de alojamiento de repositorios basada en Git.

A su vez, hablamos de commits, elementos necesarios para los cambios que se realizan en el proyecto. Un tema importante, son los Repositorios remotos junto con Push y Pull, como operaciones para enviar y recibir cambios entre un repositorio local y un repositorio remoto. También trabajamos con ramas, merges, y Pull Requests.

Estos son los elementos clave relacionados con Git y GitHub que veremos a continuación. Cada uno de ellos tiene su importancia y funcionalidades específicas en el contexto de la gestión de versiones y el desarrollo de software colaborativo.

Aprendizaje esperado

Al finalizar este curso serás capaz de:

- Conocer los fundamentos de GIT y GitHub Fundamentos de GIT
- Entender la necesidad de un repositorio de código fuente Instalación, configuración y comandos básicos Commits y restauración de archivos
- Realizar cambios de nombres
- Saber cómo ignorar archivos
- Trabajar con ramas, uniones, conflictos y tags
- Trabajar con repositorios remotos, Push y Pull Fetch v/s Pull
- Clonar un repositorio
- Administrar Pull Request
- Trabajar con flujos de trabajo con GitHub y Git
- Configurar del VCS en Visual Studio Code
- Configurar del VCS en IntelliJ IDEA

Fundamentos de GIT

Es un sistema de control de versiones distribuido que permite a los desarrolladores gestionar y colaborar en proyectos de software de manera eficiente, **manteniendo un historial completo de los cambios** realizados en los archivos del proyecto.

Se utiliza a través de la línea de comandos, pero también existen interfaces gráficas y herramientas de terceros que facilitan su uso. Algunas de las plataformas populares que aprovechan Git para la gestión de proyectos son GitHub, GitLab y Bitbucket.

GitHub

GitHub es una plataforma web basada en Git que facilita la colaboración y el almacenamiento de proyectos de desarrollo de software. **Es uno de los servicios más populares para alojar repositorios Git y proporciona herramientas adicionales para la gestión de proyectos y la colaboración en equipo.**

Algunas de las características clave de GitHub incluyen:

→ Control de versiones

GitHub aprovecha la potencia de Git para proporcionar un historial completo de cambios en los archivos de un proyecto, así como la capacidad de crear ramas, fusionar cambios y revertir revisiones.

→ Colaboración

Los desarrolladores pueden colaborar en proyectos compartiendo sus repositorios con otros y permitiendo la contribución de código a través de solicitudes de extracción (pull requests). Esto facilita el trabajo en equipo y la revisión de cambios antes de fusionarlos en el proyecto principal.

→ Problemas y seguimiento de tareas

GitHub permite la creación de problemas (issues) para realizar un seguimiento de errores, solicitudes de nuevas características y otras tareas relacionadas con el proyecto. Los problemas pueden asignarse a miembros del equipo, etiquetarse y comentarse para facilitar la comunicación.

→ Integración continua

GitHub proporciona integración con herramientas de integración continua (CI) como Travis CI, CircleCI o Jenkins. Esto permite automatizar la ejecución de pruebas, compilación de código y otras tareas durante el desarrollo del proyecto.

→ Despliegue

Con las funcionalidades de GitHub Actions, se pueden automatizar también las tareas de despliegue del software en diferentes entornos, como servidores de producción o plataformas de servicios en la nube.

GitHub es una plataforma de alojamiento de repositorios basados en Git que fomenta la colaboración y proporciona herramientas adicionales para la gestión de proyectos de desarrollo de software. Es ampliamente utilizado por la comunidad de desarrolladores para compartir, colaborar y mantener el control de versiones de sus proyectos.

Necesidad de un repositorio de código fuente

Un repositorio de código fuente es una herramienta fundamental para el desarrollo de aplicaciones por varias razones:

Control de versiones

Un repositorio de código fuente, como Git (que puede estar alojado en plataformas como GitHub), permite realizar un seguimiento de los cambios realizados en el código fuente de la aplicación a lo largo del tiempo. Esto es importante para mantener un historial completo de revisiones, facilitar la colaboración en equipo y revertir cambios si es necesario.

Respaldo y seguridad

Al utilizar un repositorio de código fuente, se mantiene una copia segura y respaldada de todo el código fuente de la aplicación. Esto protege contra la pérdida de datos en caso de fallos en los equipos locales, ya que el código se encuentra almacenado en un servidor remoto. Además, se pueden configurar copias de seguridad periódicas y almacenamiento en la nube para mayor seguridad.

Despliegue y versionado

Un repositorio de código fuente permite etiquetar versiones específicas del código fuente de la aplicación, lo que facilita el despliegue y la distribución de versiones estables. También se pueden configurar flujos de trabajo automatizados, como integración continua (CI) y entrega continua (CD), para automatizar tareas como la compilación, las pruebas y el despliegue de la aplicación.

Colaboración abierta y comunidad

Utilizar un repositorio de código fuente público, como GitHub, fomenta la colaboración abierta y la participación de la comunidad. Permite compartir el código fuente de la aplicación con otros desarrolladores, recibir aportes, comentarios y mejoras, así como contribuir a proyectos de código abierto.

Para concluir, un repositorio de código fuente es esencial en Android Studio para gestionar de manera eficiente el desarrollo de la aplicación, facilitar la colaboración en equipo, mantener un historial de cambios, garantizar la seguridad y respaldo del código, y aprovechar herramientas de despliegue y automatización.

Instalación, configuración y comandos básicos

Para poder vincular un repositorio de GitHub, comencemos creando uno:

1. Primero Inicia sesión en tu cuenta de GitHub o crea una nueva si no tienes una. Haz clic en el botón "New" o "Nuevo" para crear un nuevo repositorio.
2. Ahora, asigna un nombre al repositorio, opcionalmente añade una descripción y elige si quieres que sea público o privado.
3. Haz clic en "**Create repository**" o "Crear repositorio" para crearlo.

Instalación de GIT:

Paso 1: Descargar el instalador de Git Bash:

Ve al sitio web oficial de Git (<https://git-scm.com/>) y busca el botón de descarga. Git es compatible con diferentes sistemas operativos, así que asegúrate de seleccionar la versión correcta para tu sistema.

Paso 2: Ejecutar el instalador:

Una vez que hayas descargado el instalador, ábrelo haciendo doble clic en el archivo descargado.

Paso 3: Configurar el instalador:

Aparecerá un asistente de configuración para la instalación de Git. Asegúrate de leer y aceptar los términos de licencia.

Paso 4: Seleccionar componentes:

En esta etapa, puedes elegir los componentes que deseas instalar. Asegúrate de seleccionar "Git Bash Here" o "Git Bash Here & GUI" para asegurarte de que Git Bash esté disponible en el menú contextual del Explorador de archivos (Windows).

Paso 5: Elegir la ubicación de instalación:

Elige la ubicación donde deseas instalar Git. La configuración predeterminada suele ser adecuada para la mayoría de los usuarios.

Paso 6: Seleccionar el editor de texto (opcional):

El instalador puede pedirte que elijas un editor de texto para usar con Git Bash. Puedes seleccionar el que prefieras o dejar el predeterminado.

Paso 7: Añadir Git al PATH:

Esta opción permitirá que puedas acceder a Git desde la línea de comandos (cmd) o desde otras aplicaciones sin tener que especificar la ruta completa. Es recomendable seleccionar "Git from the command line and also from 3rd-party software".

Paso 8: Configurar la terminal predeterminada:

Puedes elegir la terminal que prefieras utilizar con Git Bash. Si tienes Visual Studio Code instalado, puedes seleccionar "Use Windows' default console window" o "Use MinTTY" para utilizar una terminal más avanzada.

Paso 9: Configuración adicional:

En esta etapa, se te pedirá que configures el comportamiento de finalización de línea. La configuración predeterminada suele ser adecuada para la mayoría de los usuarios.

Paso 10: Instalación:

Una vez que hayas revisado todas las configuraciones, haz clic en "Install" para iniciar la instalación.

Paso 11: Finalizar la instalación:

Una vez que se haya completado la instalación, haz clic en "Finish" para cerrar el asistente de configuración.

Ya tenemos nuestro repositorio, ahora ya podemos configurar Git en nuestra consola de GIT BASH:

Para abrir Git Bash, sigue estos pasos:

En Windows:

4. Ve al menú Inicio o a la pantalla de inicio de Windows.
5. Busca "Git Bash" en el cuadro de búsqueda.
6. Haz clic en el icono "Git Bash" que aparece en los resultados de búsqueda.

En macOS:

1. Abre Finder.
2. Navega a la carpeta "Aplicaciones" (Applications).
3. Busca la carpeta "Utilities" y ábrela.
4. Busca el programa "Terminal" y ábrelo.
5. Una vez que la Terminal esté abierta, Git Bash debería estar disponible si ya has instalado Git.

Una vez que hayas abierto Git Bash, verás una ventana de línea de comandos similar a la terminal. A partir de este punto, puedes comenzar a utilizar Git Bash para interactuar con Git y GitHub a través de la línea de comandos.

Pasos iniciales que puedes seguir:**1. Verificar la instalación de Git:**

Para asegurarte de que Git se haya instalado correctamente, puedes ejecutar el siguiente comando en Git Bash:

```
git --version
```

Si la instalación es exitosa, verás la versión de Git que tienes instalada.

2. Configurar tu nombre de usuario y dirección de correo electrónico:

Es importante configurar tu nombre de usuario y dirección de correo electrónico en Git para que tus commits reflejen correctamente tu identidad. Utiliza los siguientes comandos, reemplazando "Tu Nombre" y "tu@email.com" con tus datos reales:

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tu@email.com"
```

3. Clonar un repositorio desde GitHub:

Podes clonar un repositorio existente desde GitHub en tu computadora utilizando Git Bash. Ve a la página del repositorio en GitHub y copia la URL del repositorio. Luego, en Git Bash, navega a la ubicación donde deseas clonar el repositorio y ejecuta el siguiente comando, reemplazando "URL_del_repositorio" con la URL que copiaste:

```
git clone URL_del_repositorio
```

4. Crear un nuevo repositorio desde cero:

Si deseas crear un nuevo repositorio en tu computadora desde cero, simplemente navega a la ubicación donde deseas que se encuentre el repositorio y ejecuta los siguientes comandos:

```
git init
```

5. Realizar el primer commit.

Para lograrlo, añade todos los archivos modificados y nuevos al área de preparación con el siguiente comando:


```
git add .
```

Realiza el commit con el siguiente comando:

```
git commit -m "Primer commit"
```

6. Subir los cambios al repositorio remoto.

Es hora de subir tus cambios al repositorio remoto en GitHub con el siguiente comando:

```
git push -u origin master
```

Esto enviará la rama master a tu repositorio remoto. Si estás utilizando una rama diferente, reemplaza **“master”** con el nombre de tu rama.

Commits y restauración de archivos

Los commits en Git son utilizados para guardar y registrar los cambios realizados en el repositorio. Te mostraré cómo realizar commits y restaurar archivos en Git.

Veamos los pasos para realizar un commits

1. Verificar el estado de los archivos: Antes de realizar un commit, puedes utilizar el siguiente comando para ver los cambios realizados en los archivos del repositorio:

```
git status
```

2. Agregar los archivos modificados al área de preparación: Utiliza el siguiente comando para agregar los archivos modificados al área de preparación (staging area):

```
git add nombre_de_tu_archivo
```

3. Realizar el commit: Una vez que los archivos están en el área de preparación, puedes realizar el commit con el siguiente comando: (Cambia "tu descripción" Por una muy breve de los cambios realizados)

```
git commit -m "Tu descripción"
```

Veamos ahora cómo restaurar archivos

1. Deshacer cambios en el directorio de trabajo:

Si deseas deshacer los cambios realizados en un archivo y volver a la versión anterior almacenada en el último commit, utiliza el siguiente comando:

```
git checkout -- nombre_de_tu_archivo
```

2. Restaurar archivos desde el área de preparación:

Si deseas quitar los archivos del área de preparación y restaurarlos a su estado anterior, utiliza el siguiente comando:

```
git reset HEAD nombre_de_tu_archivo
```

3. Restaurar un commit anterior:

Si necesitas deshacer un commit y restaurar los archivos a una versión anterior, puedes utilizar el siguiente comando:

```
git revert hash_del_commit
```

Por último, reemplaza **hash_del_commit** con el identificador único (hash) del commit al que deseas regresar. Esto creará un nuevo commit que deshace los cambios introducidos por el commit especificado.

Cambios de nombres

Uno de los cambios que podemos hacer desde git, es hacer un cambio de nombres en los archivos.

Para eso, seguiremos los siguientes pasos:

- Verificar el estado actual de los archivos, con el siguiente comando:

```
git status
```

- Renombrar el archivo utilizando el comando **git mv** seguido del nombre actual del archivo y el nuevo nombre que desees asignarle. Por ejemplo, si quieres cambiar el nombre de "archivo_viejo.txt" a "archivo_nuevo.txt", ejecuta el siguiente comando:

```
git mv archivo_antiguo.txt archivo_nuevo.txt
```

- puedes verificar los cambios realizados utilizando nuevamente el comando **git status** para verificar los cambios realizados:

```
git status
```

- Realizar un commit para agregar los cambios al área de preparación, y entre las comillas agrega una descripción del mismo:

```
git add .  
git commit -m "mje descriptivo para el commit"
```

Después de seguir estos pasos, el archivo se habrá renombrado en tu repositorio Git. Hay que tener en cuenta que el cambio de nombre se verá reflejado en todos los commits posteriores y en las versiones futuras del archivo.

Ignorando archivos

Otra acción disponible en Git, es la de ignorar archivos, ya que algunos pueden no ser necesarios para el proyecto.

Para ignorar archivos en Git, puedes crear un archivo llamado **.gitignore** en el directorio raíz de tu repositorio y agregar los patrones de archivos que deseas ignorar. Estos patrones ayudan a especificar los archivos o tipos de archivos que no deseas que se rastreen o incluyan en el control de versiones.

Veamos cuales son los pasos para ignorar archivos en Git:

1. Crea un archivo `.gitignore` en el directorio de tu proyecto. Podes hacerlo a través de la línea de comandos o utilizando el explorador de archivos del sistema operativo.
2. Abre el archivo `.gitignore` en un editor de texto y agrega los patrones de archivos que deseas ignorar. Cada uno de estos patrones se colocan en una línea separada. Puedes utilizar comodines y reglas específicas para los patrones. }

Por ejemplo, si deseas ignorar todos los archivos `.txt` y el directorio `logs`, puedes agregar estas líneas al archivo `.gitignore`:

```
*.txt  
logs/
```

Esto evitará que se rastreen todos los archivos con extensión `.txt` y el directorio `logs`.

3. Guarda el archivo `.gitignore` y ciérralo.

Asegúrate de no haber realizado el seguimiento de los archivos o directorios que deseas ignorar antes de agregar el archivo **.gitignore**. Puedes utilizar el siguiente comando para verificar el estado actual de los archivos:

```
git status
```

4. Realiza un commit que incluya el archivo **.gitignore**:

```
git add .gitignore  
git commit -m "Agregado archivo .gitignore"
```

Una vez completos estos pasos, Git comenzará a ignorar los archivos y directorios especificados en el archivo `.gitignore`. Estos archivos no serán rastreados y no se incluirán en los commits futuros.

Ramas, uniones, conflictos y tags

Las ramas “**branches**”, las fusiones “**merges**”, los conflictos y las etiquetas “**tags**” son conceptos importantes en Git para administrar el flujo de trabajo y organizar los commits. Veamos cada uno de ellos:

Ramas (**branches**)

Las ramas en Git son líneas de desarrollo independientes que permiten trabajar en diferentes características o solucionar problemas sin afectar la rama principal, la cual, mayormente se nombra como **master**.

Cada rama tiene su historial de commits y cambios específicos, lo que facilita el desarrollo paralelo y la organización del trabajo en equipo.

Para crear una rama en git:

- Comenzaremos llamando a la nueva rama, en nuestro caso se llama “nombre-rama”:

```
git branch nombre-rama
```

- Ahora, nos movemos hacia la nueva rama:

```
git checkout nombre-rama
```

- Haz los cambios necesarios en los archivos de tu proyecto mientras te encuentras en la rama “nueva-rama”. Por ejemplo, puedes agregar un nuevo archivo o modificar el código existente.
- Realiza un commit en a nueva rama creada:

```
git add .  
git commit -m "Agregado archivo y cambios en la nueva rama"
```

- Vuelve a la rama principal (por ejemplo, "master"):

```
git checkout master
```

- Por último, es momento de fusionar la rama "nombre-rama" en la rama principal:

```
git merge nombre-rama
```

Fusiones (merges):

Las fusiones se utilizan para combinar los cambios de una rama en otra rama, generalmente para unir una rama de desarrollo a la rama principal.

Podes notar, que en el ejemplo anterior, utilizamos una fusión en el último paso, fusionando la nueva rama creada con el menú principal. Por lo que ya tienes la demostración del funcionamiento.

Conflictos

Los conflictos ocurren cuando Git no puede combinar automáticamente los cambios de dos ramas debido a modificaciones conflictivas en las mismas líneas de código.

Git indicará los archivos con conflictos y te mostrará los cambios en conflicto. Debes resolver los conflictos manualmente editando los archivos afectados y eligiendo qué cambios mantener.

Una vez que se resuelven los conflictos, debes realizar un commit para finalizar la fusión.

Etiquetas (tags):

Las etiquetas son punteros inmutables a un commit específico en la historia del repositorio. Los cuales se utilizan para marcar versiones o hitos importantes. Y son muy útiles para marcar versiones estables de tu proyecto.

Veamos con un ejemplo, el modo de trabajar con etiquetas:

Antes que nada, debemos verificar los commits existentes. Para eso utiliza el siguiente comando para ver el historial de commits en tu repositorio:

```
git log --oneline
```

Esto mostrará una lista de commits con sus identificadores únicos “**hash**” y mensajes descriptivos.

Es momento de crear una etiqueta ligera (**lightweight tag**):

Puedes crear una etiqueta ligera en un commit específico utilizando el siguiente comando:

```
git tag nombre_etiqueta commit_hash
```

Cambia “nombre_etiqueta” por el nombre que quieras usar y agrega **commit_hash** con el identificador único del commit al que deseas etiquetar.

```
git tag v1.0 abcdef123456
```

Esto creará una etiqueta llamada "v1.0" en el commit con el identificador "abcdef123456".

Encontrarás más tipos de etiquetas, y recuerda que son muy útiles para marcar versiones o hitos importantes en tu proyecto. Podes utilizar comandos adicionales para eliminar etiquetas, compartirlas en un repositorio remoto y realizar otras operaciones relacionadas con las etiquetas según tus necesidades.

Repositorios remotos, Push y Pull

Fetch v/s Pull

Los **repositorios remotos**, y los comandos **push**, **pull**, **fetch** y **pull** son conceptos y comandos importantes en Git y GitHub para colaborar con otros desarrolladores y mantener sincronizados los cambios en un proyecto. Veamos un poco de cada uno:

Repositorios remotos:

Un repositorio remoto no es más que una copia del repositorio Git en un servidor remoto, como GitHub, GitLab o Bitbucket. El cual permite colaborar con otros desarrolladores, compartir el código y realizar un seguimiento de los cambios en un proyecto de manera centralizada.

Una gran ventaja es que se puede clonar un repositorio remoto en tu máquina local utilizando el comando **git clone**. Por ejemplo:

```
git clone https://github.com/usuario/turepo.git
```

Recuerda que debes cambiar `https://github.com/usuario/turepo.git` por la url de tu proyecto.

Después de clonar, tendrás una copia local del repositorio y podrás realizar cambios y commits en tu máquina.

push:

El comando **push** se utiliza para enviar los commits locales al repositorio remoto. Lo que permite compartir tus cambios con otros desarrolladores y mantener sincronizado el repositorio remoto con tus últimas modificaciones.

Utiliza el comando `git push` seguido del nombre de la rama y el repositorio remoto al que deseas enviar los cambios.

Por ejemplo, `git push origin master` enviará los commits de la rama "master" al repositorio remoto llamado "origin".

Siguiendo el ejemplo que arrancamos clonando nuestro repositorio, utilizaremos **push** para que los cambios realizados se guarden en el repositorio remoto. Pero ahora, además de clonar, vamos a crearle algunos cambios, para que sean guardados utilizando push

- Ya tenemos esto, del paso anterior:

```
git clone <URL_de_tu_repositorio>
```

- Puedes realizar cambios en el archivo:

Navega al directorio del repositorio clonado y realiza los cambios deseados en un archivo específico utilizando el editor de texto o el IDE de tu elección.

- Verificar el estado de los cambios:

Utiliza el comando `git status` para verificar el estado de los cambios en tu repositorio local. Deberías ver el archivo modificado listado como un archivo sin seguimiento.

- Agregar los cambios al área de preparación:

Utiliza el comando **git add** seguido del nombre del archivo modificado para agregar los cambios al área de preparación (**staging area**). Por ejemplo:

```
git add nombre_del_archivo_modificado
```

- Realizar el commit:

Realiza un commit para guardar los cambios en el repositorio local. Utiliza el comando `git commit -m` seguido de un mensaje descriptivo para el commit. Por ejemplo:

```
git commit -m "Agregados cambios en el archivo"
```

- Realizar el push:

Utiliza el comando `git push` seguido del nombre del repositorio remoto y la rama a la que deseas enviar tus cambios. Por ejemplo:

```
git push origin master
```

Reemplaza **origin** con el nombre del repositorio remoto (generalmente es origin si clonaste el repositorio) y **master** con el nombre de la rama en la que deseas realizar el push.

Opcional:

Si es la primera vez que haces un push en esta rama, es posible que necesites agregar la opción **-u** para establecer la rama **upstream**:

```
git push -u origin master
```

- Proporciona tus credenciales:

Si es la primera vez que realizas un push a este repositorio remoto o si se requiere autenticación, se te solicitará ingresar tus credenciales de GitHub (nombre de usuario y contraseña o token de acceso personal).

- Espera a que finalice el push:

Git enviará tus cambios al repositorio remoto en GitHub. Una vez que el push se complete, verás un mensaje indicando que los cambios se han enviado correctamente.

Ahora ya podrás ver que tus cambios locales se han enviado al repositorio remoto en GitHub. Otros colaboradores también podrán ver tus modificaciones y trabajar con ellas. Recuerda realizar push cada vez que desees compartir tus cambios y mantener sincronizado el repositorio remoto con tus últimas modificaciones.

pull:

El comando pull se utiliza para recuperar los cambios del repositorio remoto y fusionarlos con tu copia local. Este comando se combina con el comando **fetch** (lo veremos a continuación) que es para recuperar y con el comando **merge** que es para fusionar (este ya lo trabajamos anteriormente) en un solo paso.

Utiliza el comando git pull seguido del nombre de la rama y el repositorio remoto desde donde desees obtener los cambios.

Por ejemplo, **git pull origin master** recuperará los cambios de la rama "master" del repositorio remoto llamado "origin" y los fusionará automáticamente en tu rama local actual.

Aquí tienes un ejemplo de cómo realizar un pull para obtener los últimos cambios de un repositorio remoto y fusionarlos con tu repositorio local:

- **Verificar el estado de tus cambios locales:**

Antes de realizar un pull, es recomendable verificar el estado de tus cambios locales. Utiliza el comando git status para asegurarte de que no tengas cambios sin confirmar.

- **Realizar un pull desde el repositorio remoto:**

Utiliza el comando git pull seguido del nombre del repositorio remoto y la rama de la cual desees obtener los cambios. Por ejemplo:

```
git pull origin master
```

Reemplaza **origin** con el nombre del repositorio remoto (generalmente es **origin** si clonamos el repositorio) y master con el nombre de la rama desde la cual deseas obtener los cambios.

- **Proporcionar sus credenciales:**

Si se requiere autenticación para acceder al repositorio remoto, es posible que se te solicite ingresar tus credenciales de GitHub (nombre de usuario y contraseña o token de acceso personal).

- **Esperar a que finalice el pull:**

Git recuperará los últimos cambios del repositorio remoto y los fusionará automáticamente en tu rama local. Si no hay conflictos, el pull se completará sin problemas.

En caso de que haya conflictos, Git te notificará y mostrará los archivos afectados. Deberás resolver los conflictos manualmente editando los archivos en conflicto y luego hacer un commit para finalizar la fusión.

fetch vs pull en GitHub

En GitHub, **fetch** se utiliza para obtener los cambios del repositorio remoto sin fusionar automáticamente los cambios en tu rama local.

Con fetch, los cambios se descargan a tu repositorio local, pero no se fusionan automáticamente, lo que te permite revisar los cambios antes de incorporarlos.

Para fusionar los cambios después de realizar fetch, puedes utilizar el comando `git merge` o `git rebase` para combinar los cambios descargados con tu rama local.

Por otro lado, pull en GitHub es similar a fetch pero automáticamente fusiona los cambios descargados en tu rama local sin necesidad de ejecutar un comando de fusión adicional.

Si estás trabajando en un proyecto en solitario o en una rama dedicada, pull puede ser conveniente. Sin embargo, si estás colaborando con otros desarrolladores y deseas revisar los cambios antes de fusionarlos, puedes optar por fetch y luego realizar la fusión manualmente.

Recuerda que estos comandos son esenciales para colaborar y mantener sincronizado tu repositorio local con el repositorio remoto en GitHub u otras plataformas de alojamiento de repositorios. Utiliza los comandos adecuados según tus necesidades y el flujo de trabajo de tu proyecto.

Administrando Pull Request

¿Qué es un pull request?

Un **pull request** o **solicitud de extracción** es una función clave en plataformas de alojamiento de repositorios como GitHub. Es un mecanismo que permite a los desarrolladores colaborar y revisar los cambios realizados en un repositorio antes de que sean fusionados en la rama principal o en otra rama.

Podemos verlo como una solicitud que un desarrollador envía al propietario o colaboradores del repositorio para que revisen y consideren los cambios propuestos.

El flujo típico de trabajo con pull requests es el siguiente:

- Un desarrollador crea una rama separada en el repositorio para trabajar en una nueva función, solucionar un error o realizar cualquier otro tipo de modificación.
- Después de realizar los cambios y realizar los commits en su rama, el desarrollador envía un pull request al repositorio principal o a la rama base a la que desea fusionar sus cambios.
- Otros colaboradores pueden revisar el pull request, ver los cambios realizados, hacer comentarios y sugerencias, y solicitar modificaciones adicionales si es necesario.
- El autor del pull request puede realizar las modificaciones solicitadas, agregar más commits a su rama y actualizar automáticamente el pull request para que los revisores vean los cambios actualizados.
- Una vez que los cambios en el pull request han sido aprobados y revisados adecuadamente, el propietario del repositorio o un colaborador con permisos suficientes puede fusionar los cambios en la rama base.

Los pull requests **son valiosos en entornos colaborativos, ya que permiten una revisión de código más rigurosa y fomentan la colaboración abierta entre los**

miembros del equipo. También facilitan el seguimiento de los cambios realizados, proporcionan una plataforma para discusiones y permiten una integración más controlada y organizada de los cambios en un proyecto.

Administrar un Pull Request

Administrar Pull Requests es una parte fundamental del flujo de trabajo colaborativo en GitHub. Aquí tienes algunos pasos básicos para administrar Pull Requests:

Crear un Pull Request:

1. Desde la página principal del repositorio en GitHub, haz clic en el botón "New pull request" (Nuevo pull request).
2. Selecciona la rama de la cual deseas fusionar los cambios (rama base) y la rama que contiene los cambios que deseas fusionar (rama comparada).
3. Proporciona un título descriptivo y una descripción detallada de los cambios realizados en la rama comparada.
4. Revisa los cambios y, si es necesario, agrega comentarios o realiza modificaciones adicionales.
5. Finalmente, haz clic en el botón "Create pull request" (Crear pull request) para enviar el Pull Request.

Revisar y comentar en el Pull Request:

Otros colaboradores pueden revisar tus cambios y dejar comentarios en el Pull Request.

Puedes responder a los comentarios, hacer modificaciones adicionales o discutir los cambios propuestos antes de que se fusionen.

Fusionar el Pull Request:

Una vez que los cambios en el Pull Request hayan sido revisados y aprobados, puedes fusionarlos en la rama base.

1. En la página del Pull Request, haz clic en el botón "Merge pull request" (Fusionar pull request) para incorporar los cambios a la rama base.

2. Puedes seleccionar opciones adicionales, como "Squash and merge" (Fusionar y comprimir) o "Rebase and merge" (Fusionar y rebase), dependiendo del flujo de trabajo preferido.
3. Confirma la fusión del Pull Request.
4. Cerrar el Pull Request

Después de fusionar los cambios, puedes cerrar el Pull Request para indicar que se ha completado.

En la página del Pull Request, haz clic en el botón "**Close pull request**" (Cerrar pull request) para cerrarlo.

Estos son los pasos básicos para administrar Pull Requests en GitHub. Recuerda que el flujo de trabajo puede variar según las preferencias y políticas del equipo. Además, GitHub proporciona características adicionales, como la posibilidad de asignar revisores, integración continua y otras opciones avanzadas para mejorar la colaboración en el desarrollo de proyectos.

Configuración de los editores de código

¿Por qué es importante configurar el VCS?

Configurar el sistema de control de versiones (VCS), como Git, es importante por varias razones:

1. Gestión de versiones

El VCS te permite realizar un seguimiento de los cambios realizados en tu proyecto a lo largo del tiempo. Puedes ver el historial de cambios, quién hizo cada cambio y revertir a versiones anteriores si es necesario. Esto es útil para mantener un registro de las modificaciones y facilitar la colaboración en equipos de desarrollo.

2. Colaboración en equipo

Configurar el VCS en los editores de código te permite colaborar eficientemente con otros desarrolladores en un proyecto. Puedes trabajar en paralelo en diferentes ramas, fusionar cambios y resolver conflictos de manera efectiva.

Además, puedes revisar y comentar los cambios propuestos por otros miembros del equipo antes de fusionarlos en la rama principal.

3. Seguridad y respaldo

Al utilizar un VCS, como Git, tus cambios se almacenan de forma segura en el repositorio remoto. Esto proporciona un respaldo para tu código y evita la pérdida de trabajo en caso de fallos o errores. Además, puedes configurar reglas de acceso y permisos para proteger tu repositorio y controlar quién puede realizar cambios y revisiones.

4. Gestión de ramas y características

El VCS te permite crear ramas separadas para trabajar en nuevas características, solucionar errores o experimentar con cambios. Puedes trabajar en una rama sin afectar directamente la rama principal y fusionar los cambios cuando estén listos. Esto facilita la gestión de diferentes versiones de tu proyecto y te permite probar y desarrollar nuevas funcionalidades de forma aislada.

5. Historial y análisis de cambio

Configurar el VCS te permite analizar el historial de cambios en tu proyecto, lo que puede ser útil para comprender cómo ha evolucionado tu código, identificar y solucionar problemas, y realizar análisis de rendimiento. Puedes ver quién hizo cada cambio, cuándo y por qué, lo que facilita el seguimiento de la evolución del proyecto.

Para finalizar este apartado, entendemos que configurar el VCS en los editores de código es importante para una gestión eficiente de versiones, una colaboración efectiva en equipo, la seguridad de tu código y la capacidad de realizar un seguimiento de los cambios y realizar análisis en tu proyecto. Facilita un flujo de trabajo ordenado y mejora la productividad en el desarrollo de software.

¿Cómo configurarlo?

La configuración del sistema de control de versiones (VCS, por sus siglas en inglés) en los editores de código te permite integrar y utilizar Git para gestionar tu proyecto.

Acá tenes los pasos básicos para configurar el VCS en los editores de código:

Iniciar un repositorio usando Visual Studio Code

Para configurar Git y GitHub en Visual Studio Code, sigue estos pasos:

1. Instalar Visual Studio Code:

Si no tienes Visual Studio Code, puedes descargarlo e instalarlo desde el sitio oficial: <https://code.visualstudio.com/>

2. Abrir Visual Studio Code:

Una vez que hayas instalado Visual Studio Code, ábrelo en tu computadora.

3. Instalar la extensión de Git en Visual Studio Code:

Haz clic en el ícono de "Extensions" (cuarta opción en la barra lateral vertical, o simplemente usa el atajo Ctrl + Shift + X). Busca "Git" en el cuadro de búsqueda y selecciona "Git" de la lista de resultados. Luego, haz clic en el botón "Install" para instalar la extensión.

4. Configurar Git en Visual Studio Code:

Asegúrate de que Visual Studio Code esté configurado para usar Git correctamente. Para ello, sigue estos pasos:

- Abre Visual Studio Code y navega a "View" (Vista) en la barra de menú.
- Selecciona "Command Palette..." (Paleta de comandos) o usa el atajo Ctrl + Shift + P.
- Escribe "Git: Set Global Git Configuration" en el cuadro de búsqueda y selecciona la opción.
- Se te pedirá que ingreses tu nombre de usuario de GitHub y tu dirección de correo electrónico. Esto configurará Git globalmente en tu computadora.

5. Clonar un repositorio desde GitHub:

- Ve a la página del repositorio en GitHub que deseas clonar en Visual Studio Code.
- Haz clic en el botón verde "Code" y copia la URL del repositorio.
- Vuelve a Visual Studio Code y selecciona "View" (Vista) en la barra de menú.

- Elige "Command Palette..." (Paleta de comandos) o usa el atajo Ctrl + Shift + P.
- Escribe "Git: Clone" en el cuadro de búsqueda y selecciona la opción.
- Pega la URL del repositorio que copiaste previamente y selecciona una ubicación para clonar el repositorio en tu computadora.

6. Configurar GitHub en Visual Studio Code:

Para que puedas trabajar con GitHub directamente desde Visual Studio Code, necesitarás configurar la integración de GitHub. Sigue estos pasos:

- Ve a "View" (Vista) en la barra de menú de Visual Studio Code.
- Selecciona "Command Palette..." (Paleta de comandos) o usa el atajo Ctrl + Shift + P.
- Escribe "Sign in to GitHub" en el cuadro de búsqueda y selecciona la opción.
- Se abrirá una ventana emergente solicitando tu autorización para acceder a GitHub. Haz clic en "Sign in with your browser" (Iniciar sesión con tu navegador) y sigue las instrucciones para iniciar sesión en tu cuenta de GitHub.
- Una vez que hayas iniciado sesión, recibirás un código de autorización. Copia el código y pégalo en la ventana emergente de Visual Studio Code.

7. ¡Listo para usar!

Ahora que has configurado Git y GitHub en Visual Studio Code, podrás clonar, crear, editar y gestionar tus repositorios directamente desde el entorno de desarrollo.

Iniciar un repositorio usando IntelliJ IDEA

En IntelliJ IDEA, también puedes configurar Git y GitHub para gestionar tus repositorios y colaborar en proyectos. A continuación, te mostraré cómo hacerlo:

1. Instalar IntelliJ IDEA:

Si aún no tienes IntelliJ IDEA, descárgalo e instálalo desde el sitio oficial: <https://www.jetbrains.com/idea/>

2. Configurar Git en IntelliJ IDEA:

Una vez que tengas Git instalado, sigue estos pasos para configurarlo en IntelliJ IDEA:

- Abre IntelliJ IDEA en tu computadora.
- Ve a "File" (Archivo) en la barra de menú y selecciona "Settings" (Configuración) o usa el atajo Ctrl + Alt + S.
- En la ventana de configuración, busca "Version Control" (Control de versiones) en el panel izquierdo y selecciona "Git" en la lista desplegable.
- Asegúrate de que el campo "Path to Git executable" (Ruta al ejecutable de Git) apunte al archivo ejecutable de Git en tu computadora. Si Git se instaló correctamente, IntelliJ IDEA debería detectar automáticamente la ruta. Si no, deberás proporcionar la ruta manualmente.

3. Configurar GitHub en IntelliJ IDEA:

Para configurar GitHub en IntelliJ IDEA y trabajar con repositorios remotos, sigue estos pasos:

- Ve a "File" (Archivo) en la barra de menú y selecciona "Settings" (Configuración) o usa el atajo Ctrl + Alt + S.
- En la ventana de configuración, busca "Version Control" (Control de versiones) en el panel izquierdo y selecciona "GitHub" en la lista desplegable.
- Haz clic en "Configure GitHub" y sigue las instrucciones para iniciar sesión en tu cuenta de GitHub y autorizar IntelliJ IDEA para acceder a tus repositorios.

4. Clonar un repositorio desde GitHub:

Para clonar un repositorio desde GitHub en IntelliJ IDEA, sigue estos pasos:

- Ve a "VCS" en la barra de menú y selecciona "Get from Version Control" (Obtener desde el control de versiones) o usa el atajo Ctrl + Alt + V.
- Pega la URL del repositorio de GitHub en el campo "URL" y elige la ubicación donde deseas clonar el repositorio en tu computadora.
- Haz clic en "Clone" para clonar el repositorio.

5. Crear, editar y gestionar archivos:

Una vez que hayas clonado el repositorio, podrás crear, editar y gestionar archivos en IntelliJ IDEA. Los cambios que realices se reflejarán en tu repositorio local.

6. Hacer commits y Push:

Para hacer commits de tus cambios y enviarlos al repositorio remoto en GitHub, sigue estos pasos:

- Haz clic derecho en el archivo o en la carpeta que desees commitear y selecciona "Git" en el menú contextual.
- Elige "Commit File" para commitear un archivo específico o "Commit Directory" para commitear todos los cambios en una carpeta.
- Escribe un mensaje de commit descriptivo y haz clic en "Commit".
- Luego, para enviar tus commits al repositorio remoto en GitHub, ve a "VCS" en la barra de menú y selecciona "Git" > "Push" o usa el atajo Ctrl + Shift + K.

¡Listo! Ahora has configurado Git y GitHub en IntelliJ IDEA y estás listo para gestionar tus repositorios y colaborar en proyectos desde el entorno de desarrollo. Disfruta de tu experiencia de desarrollo con IntelliJ IDEA y Git/GitHub.

Cierre

Hemos llegado al final de la lección donde trabajamos los fundamentos de GIT y GITHUB. Podemos decir, que la gestión de versiones y el control de cambios son aspectos esenciales en el desarrollo de software colaborativo. Git se ha convertido en el sistema de control de versiones más utilizado, mientras que GitHub proporciona una plataforma popular para alojar y colaborar en repositorios Git.

En el contexto de los editores de código, la integración de Git y GitHub permite a los desarrolladores aprovechar las herramientas de control de versiones integradas para realizar operaciones como commits, pushes, pulls, creación de ramas y fusiones.

Los conceptos clave que hemos visto incluyen commits, repositorios remotos, ramas, merges y Pull Requests, que son fundamentales para el seguimiento y la colaboración eficiente en proyectos de desarrollo. Además, el archivo .gitignore y la configuración del VCS en los editores de código son elementos importantes para personalizar y gestionar adecuadamente el control de versiones en un proyecto.

Para finalizar podemos concluir que comprender y utilizar efectivamente Git, GitHub y las herramientas de control de versiones en los editores de código son habilidades valiosas para cualquier desarrollador de software colaborativo. Estas herramientas y conceptos te permiten gestionar cambios, colaborar con otros desarrolladores, mantener un historial de versiones y trabajar de manera más organizada en tus proyectos de desarrollo.

Referencias

Git SCM. (s.f.). Documentación de Git. <https://git-scm.com/doc>

GitHub. (s.f.). Documentación de GitHub. <https://docs.github.com/>

JetBrains. (s.f.). Documentación de IntelliJ IDEA.
<https://www.jetbrains.com/idea/resources/>

Microsoft. (s.f.). Documentación de Visual Studio Code. <https://code.visualstudio.com/docs>

¡Muchas gracias!

Nos vemos en la próxima lección

